

Decorator Timing

Implement a Python decorator called `timeit` that measures the execution time of a function. The decorator takes a single argument, `func`, which is the function to be timed.

When the decorated function is called, the decorator should: 1. Measure the execution time in milliseconds 2. Push the execution time to a list called `execution_time` (already declared in the global scope) 3. Return the same value that the original function would return

Example

```
function_name = "delay_max"
args = [3, 4, 1]
time_delay = 150
```

Decorate the function `delay_max` with `timeit`, and call the decorated function:

```
decorated_func = timeit(delay_max)
print(decorated_func(3, 4, 1, delay=150))
```

The code should output 4, which is the return value of `delay_max(3, 4, 1, delay=150)`. Since the function `delay_max` takes 150 ms to complete, the decorator should push a value close to 150 to the `execution_time` list.

Function Description Complete the function `timeit` in the editor below. The function is a decorator with the above properties.

`timeit` has the following parameter: * **func**: a function

Constraints * $1 \leq n \leq 20$ * $1 \leq \text{args}[i][j] \leq 100$ * `function_name[i] = {"delay_max", "delay_min", "delay_sum"}` * $50 \leq \text{time_delay}[i] \leq 400$ * `time_delay[i]` is a multiple of 50.

Input Format For Custom Testing The first line contains an integer, `n`, denoting the number of function calls. Each line `i` of the `n` subsequent lines (where $0 \leq i < n$) contains space-separated values denoting `function_name[i]`, `args[i]`, and `time_delay[i]`.

Sample Case 0 Sample Input For Custom Testing

```
3
delay_max 2 3 1 100
delay_sum 2 2 2 200
delay_min 4 5 5 300
```

Sample Output

```
3
6
4
[100, 200, 300]
```

Explanation Here, there are 3 function calls * function_name = "delay_max", args = [2, 3, 1], time_delay = 100, decorate the function delay_max with timeit, and call the decorated function with arguments 2, 3, and 1 with delay = 100. It should return 3. The decorator should push a value close to 100 to the list `execution_time`. * function_name = "delay_sum", args = [2, 2, 2], time_delay = 200, decorate the function delay_sum with timeit, and call the decorated function with arguments 2, 2, and 2 with delay = 200. It should return 6. The decorator should push a value close to 200 to the list `execution_time`. * function_name = "delay_min", args = [4, 5, 5], time_delay = 300, we decorate the function delay_min with timeit, and call the decorated function with arguments 4, 5, and 5 with delay = 300. It should return 4. The decorator should push a value close to 300 to the list `execution_time`.

Therefore, `execution_time` $\cong$ [100, 200, 300]